

[Toybox] Impact of global struct size

Rob Landley [rob at landley.net](mailto:rob@landley.net)

Mon Jan 1 12:45:43 PST 2024

- Next message (by thread): [\[Toybox\] Impact of global struct size](#)
 - Messages sorted by: [\[date\]](#) [\[thread\]](#) [\[subject\]](#) [\[author\]](#)
-

On 12/30/23 18:10, Ray Gardner wrote:

```
> I am having a bit of trouble understanding the impact of globals.  
>  
> There are the probes GLOBALS and findglobals to see what the space usage  
> is for globals. The output of both show that "this" is up to 8232 bytes,  
> due to the "ip" toy using 8232 of global space.
```

Which is in pending for a reason.

```
> The blog entry of 31 August 2023 ends with some discussion of which  
> commands take up the most global space. It says "Everything "tr" and  
> earlier is reasonably sized, and "ip" and "telnet" are in pending."
```

I sorted them by size and cut and pasted the end of the list, starting with "tr".

Commands in pending haven't been (fully) cleaned up yet, so problems in them aren't yet confirmed to be a design problem requiring heavy lifting. Most likely something easily fixable I just haven't done the janitorial work for yet.

```
> I inferred that this means commands in pending are less important here,  
> but they still seem to take up space in "this".
```

Only when enabled, which they aren't in defconfig. If you don't switch it on in config, then it doesn't get build, meaning it doesn't take up any space in the resulting toybox binary.

```
> How important is the space here? "tr" was 520 then, cksum was 1024. How  
> big is too big?
```

Mostly this is an issue for embedded systems. I doubt android's going to care.

Sorry for the delay replying, I can't figure out how to explain this without a GIANT INFODUMP of backstory. The tl;dr is your read-only data is shared between all instances of the program, but your `_writable_` data needs a separate copy for each instance of the program that's running, and that includes every global variable you MIGHT write to. The physical pages can be demand-faulted on systems with an MMU (although each fault gets rounded up to page size), but without an mmu it's `LD_BIND_NOW` and then some. See "man 8 ld.so" if you didn't get that reference...)

Ok, backstory: since 1996 modern Linux executables are stored in ELF format (Executable Linking Format, yes "ELF format" is like "ATM machine"). It's an archive format like zip or tar, except what it stores is (among other things) a list of "sections" each containing a list of "symbols". Your linker puts this together from the `.o` files produced by the compiler.

Statically linked processes have six main memory mappings, four of which are initialized by the kernel's ELF loader (`fs/binfmt_elf.c`) from sections in the ELF file, and the other two are generated at runtime. All six of these are created by the kernel during the `execve(2)` system call, mostly `l` wanders through `fs/binfmt_elf.c` (or `fs/binfmt_fdpic.c` which is kind of an ext2/ext3 thing that's MOSTLY the same with minor variants and REALLY SHOULD be the same file but isn't because kernel development became proctologically recursive some years ago).

You can look at `/proc/self/maps` (and `/proc/self/smmaps`, and `/proc/self/smmaps_rollup`) to see them for a running process (replace "self" with

any running PID, self is a symlink to your current PID). The six sections are:

- text - the executable functions: `mmap(MAP_PRIVATE, PROT_READ|PROT_EXEC)`
- rodata - const globals, string constants, etc: `mmap(MAP_PRIVATE, PROT_READ)`
- data - writeable data initialized to nonzero: `mmap(MAP_PRIVATE, PROT_WRITE)`
- bss - writeable data initialized to zero: `mmap(MAP_ANON, PROT_WRITE)`
- stack - function call stack, also contains environment data
- heap - backing store for `malloc()` and `free()`

The first three of those literally exist in the ELF file, as in it `mmap()`s a block of data out of the file at a starting offset, and the memory is thus automatically populated with data from the file. The text and rodata ones don't really care if it's `MAP_PRIVATE` or `MAP_SHARED` because they can never write anything back to the file, but the data one cares that it's `MAP_PRIVATE`: any changes stay local and do NOT get written back to the file. And the bss is an anonymous mapping so starts zeroed, the file doesn't bother wasting space on a run of zeroes when the OS can just provide that on request. (It stands for Block Starting Symbol which I assume meant something useful 40 years ago on DEC hardware.)

All four of those ELF sections (text, rodata, data, bss) are each treated as a giant struct under the covers, because that's how C thinks. Every time you reference a variable the C code goes "I have a pointer to the start of this, and I have an offset into it where this particular symbol lives within that segment, and I know the type and thus size of the variable living at that offset" every time you reference a symbol that lives there.

The remaining two memory blocks aren't part of ELF, but they're needed at runtime.

The stack is also set up by the kernel, and is funny in three ways:

1) it has environment data at the end (so all your inherited environment variables, and your `argv[]` arguments, plus an array of pointers to the start of each string which is what `char *argv[]` and `char *environ[]` actually point to. The kernel's task struct also used to live there, but these days there's a separate "kernel stack" and I'd have to look up where things physically are now and what's user visible.

2) It grows down, not up. (Except on `pa-risc`, but that's essentially extinct.) The pointer starts at the end of the stack space (well, right before the environment data), and each function call SUBTRACTS from it, and each function call adds back to it.

Your local variables are basically ANOTHER struct where "I have a register pointing to a location, and each name is an offset+type from that register" (it's how C does everything). When you make a function call, the pointer moves to fresh empty stack space so the local variables from last time aren't modified by the current function, and then when you return it moves it back.

By moving down, a function can grab however much stack space it needs in one place at the start of the function (I need X bytes, so `sp -= X`), and then that "pointer+offset" logic is going into the protected space it's reserved for itself. If the stack grew up, it would either have to SUBTRACT the offset to use space it grabbed at the start, or else hold off to protect `_this_` function's space right before each function call (duplicated code, and also using unprotected space so outside observers like debuggers would never `_really_` know how much of the stack was currently in use). The function can implement "return" as shared code where it jumps to a single "add this many bytes back to the stack pointer" instruction on the way out. Add each function call pushing the instruction pointer to the stack before jumping to the new function, and then you just "pop ip" after fixing the stack pointer and you've returned from your function.

3) The stack generally has `_two_` pointers, a "stack pointer" and a "base pointer" which I always get confused. One of them points to the start of the mapping (kinda important to keep track of where your mappings are), and the other one moves (gets subtracted from and added to and offset to access local variables).

All this is ignoring dynamic linking, in which case EACH library has those first four sections (plus a PLT and GOT which have to nest since the shared libraries are THEMSELVES dynamically linked, which is why you need to run ldd recursively when harvesting binaries, although what it does to them at runtime I try not to examine too closely after eating). There should still only be one stack and heap shared by each process though.

If you launch dozens of instances of the same program, the read only sections (text and rodata) are shared between all the instances. (This is why nommu systems needed to invent fdpic: in conventional ELF everything uses absolute addresses, which is find when you've got an MMU because each process has its own virtual address range starting at zero. (Generally libc or something will mmap() about 64k of "cannot read, cannot write, cannot execute" memory there so any attempt to dereference a NULL pointer segfaults, but other than that...))

But shared libraries need to move so they can fit around stuff. Back in the a.out days each shared library was also linked at an absolute address (just one well above zero, out of the way of most programs), meaning when putting together a system you needed a registry of what addresses were used by each library, and you'd have to supply an address range to each library you were building as part of the compiler options (or linker script or however that build did it). This sucked tremendously.

So ELF PIC (Position Independent Code) was invented, where everything is offset from a single pointer (all the segments are glued together one after the other, so each segment has a "starting offset". Register + segment offset + variable offset = location of variable). PIC was invented for shared libraries, but the security nuts immediately decided they wanted executables to work like that too (so exploit code that jumped to absolute addresses where it "knew" a function lived would stop working. Moving shared libraries around helped, but moving the executable's own code around nerfed more attack surface). So they invented PIE (Position Independent Executables) which means your executable is compiled with -fpic and then your startup code (crt0.o and friends) does some shared library setup. (This may also have involved kernel changes, I knew back around 2007 but between me forgetting and the plumbing being a moving target...)

(Note: legacy 32 bit x86 does not have enough registers, so anything that uses one more register leaves less for the rest of the code resulting in extra stack spills and such, although they were already shoving %fs and %gs or some such in there decades ago and x86-64 added 8 more and x32 is a hybrid that can still use those 8 extra in 32 bit mode... Anyway, there used to be strong reasons to skimp on register usage, and these days not so much. Outside of legacy x86, architectures with FEW registers have 16, and the rest have 32. So eating a half-dozen for "API" is acceptable.)

So NOMMU systems: processors that do not have a memory management unit are generally called "microcontrollers" instead of "microprocessors", and they use physical addresses for everything. This has some advantages: not only is it less circuitry (less power consumption), but it's more memory efficient because there are no page tables (Vitaly Wool gave a talk at ELC in 2015 where he had Linux running in 256k of SRAM, and everything else running out of ROM (well, nor flash), which was only possible because he didn't spend any memory on page tables) and easier to get hard realtime behavior because there are no soft faults...

NOMMU systems have been analogized to bacteria, fungi, and insects: there are more bacterial cells on the planet than mammal cells by a VERY LARGER MARGIN. But they're mostly invisible to the elephants and dinosaurs of the world. NOMMU systems are EVERYWHERE. Mostly Linux is too big to play in that space, but even a small slice of it is more deployments than everything else Linux does combined (yes including phones).

A MMU has an extra set of registers called a TLB (Translation Lookaside Buffer) which translates each memory access (this memory address you're using is actually THIS physical memory address). Each TLB entry is a starting address, a range, and some permission flags (this entry is good for read, write, or execute attempts). The TLB entries are all checked in parallel in a single clock cycle, so there are never many of them (the wiring increases exponentially the more you have). When a memory access the code tries to do ISN'T in the TLB, the processor

generates an interrupt ("fault" to hardware people) and execution jumps to the interrupt handler, which traverses a data structure called a "page table" to figure out what that memory access should do. It can swap out one of the TLB entries to stick in the appropriate virtual/physical translation and return from the interrupt (this is called a "soft fault"). Or it can edit the page table to allocate more physical memory (if this adds a zeroed page it's deferred allocation, if it makes a copy of previously shared memory you're now writing to it's copy-on-write). If it hasn't got enough memory to satisfy the attempted access it can evict some other physical page (writing its contents to swap), generally suspending the process and letting the scheduler pick something else to run until the write's finished and it can reuse that page. Similarly, if that page isn't available because it was swapped out (or is mmaped from a file but not present in memory) the fault handler can schedule a load from disk (or network filesystem!) and again suspend the process until that completes and the data's ready to edit the page table, fix up the TLB, and resume from the fault.

Some old processors had page fault handling in hardware. This sucked rocks, and Linus yelled at the PowerPC guys somewhat extensively about this 20 years ago. Page fault handling in software is sane, page fault handling in hardware isn't. This does mean you need to tie up a TLB entry pointing at your page table, and another pointing at your memory fault handler code, so the software can run it. (Or you stick the processor into "physical addressing" mode to bypass the TLB during the interrupt so you're running without the MMU and all memory addresses are physical addresses. Different processors do it different ways.)

A NOMMU system hasn't got a TLB. Sometimes it will have "range registers", which are similar in that they provide windows into physical memory where access is allowed. The same "start, length" logic for address matching, but all it does is allow or disallow access. (That way your userspace processes can't access each other's memory, and can't read or write kernel memory either.) But this isn't very common because the MTRR plumbing is 90% of the way to a software TLB: add a third value (an offset added to the physical address when this range matches; remember unsigned values can wrap cleanly) and the RWX access type flags (so you can match TYPE of access allowing read-only or non-executable memory), give the fault handler the ability to resume the interrupted instruction after doing fixups... and you have an MMU.

Things a NOMMU system can't do:

1) Remap addresses.

On a system with an mmu, every process can have a different mapping at address 0x1000 and each sees its own unique contents there. Without an mmu, what's in the memory is the same for everybody.

2) Collate discontinuous memory.

With an mmu, if I mmap(MAP_ANON) a megabyte long, the underlying physical pages the page table slots in may be scattered all over the place, and it's not my problem.

Without an MMU, if I want a 256k mapping I need to find 256k of contiguous physical memory (all together in one linear chunk). And if the system's been running for a while, I may have way more than 256k free but it's in scattered pieces none of which is big enough to satisfy my allocation request.

Fragmentation is a much bigger problem on NOMMU systems.

3) use swap space

It's built on top of page tables, and we haven't got those. So I can't use storage to increase my available memory: the physical memory I've got is it.

4) Demand fault

If I allocate memory, I have to have that memory NOW. I can't allocate page table space and leave the physical memory unpopulated, and then attach pages to it from the fault handler as the processor actually tries to read or write to

the memory.

If I want to `mmap()` something backed from a file, I have to load it all in during the `mmap()` call. I can't detect access and load it in just the part that was accessed from the fault handler.

I can't copy-on-write either. With an MMU, I have a writeable mapping that starts identical to another process's memory I can point the new mapping at the old one's physical pages and mark it read only, and then when they try to write have the fault handler allocate a new physical page, copy the old data, and redirect the write to the new page. But without an MMU, a separate writeable mapping is separate physical pages right from the start, it just initializes it with a copy of the data.

5) `fork()` processes.

An identical copy of this process would want to use the same addresses. If you make copies of this process's mappings, they will live at different address ranges. All the pointers in the new copy point into to the OLD copy's memory.

Instead nommu systems have to use `vfork()`, which suspends the parent until the child either calls `exec()` or `_exit()`, because both discard all the old mappings (and thus the parent can get them back). Note that changes to global variables in the child before calling `exec()` will affect the parent's data!

Often `vfork()` won't even give the child a new stack, which means values written to `_local_` variables are also visible in the parent when that resumes, AND it means the child CANNOT RETURN FROM THE FUNCTION THAT CALLED VFORK (because that will discard the current stack frame, and then the next function call will overwrite it with nonsense, so when the parent resumes Bad Things Happen).

6) Most `mmap()` flags don't work.

All `mmap()` really does on nommu is allocate physical memory ranges so other mappings won't use it. There's no protection or reorganization.

You can `mmap(MAP_ANONYMOUS)`. You can only `MAP_FIXED` if nobody else is using that address yet. Everything is always `MAP_LOCKED` and `MAP_POPULATE`.

You can `mmap(MAP_SHARED)` but `_not_` with `PROT_WRITE` (because it has no way to detect changed data needs to be written back: the MMU trick is to remove the write bit from "clean" pages, take a fault the first time you try to write to them, and let the write go through but schedule the page to be written out to disk; there's optimizations with a second "dirty" bit to reduce the number of faults taken without missing updates as the page is written to disk). And remember, read-only mappings are fully populated up front, which is expensive (and a latency spike because `mmap()` won't return until it's finished reading all the data into memory).

You can't `MAP_PRIVATE` because that does copy-on-write when you change the contents. You can `mmap(MAP_SHARED)` because "everybody sees the changes" is the default, but only read-only mappings work.

7) Dynamically grow the stack.

Again, no faults for the interrupt handler to fix up via deferred physical page allocation, so the stack size you request is the stack size the kernel allocates for you in `exec`. And it's gotta be contiguous or the process can't START, so ideally nommu systems use as small a stack size as possible. (This is another extra field `fdpic` and `binflt` added: `binflt` as the old nommu variant of `a.out`, it's obsolete, don't go there. You can set it via a linker option, or it's got a default you can specify in the `./configure` when you build your toolchain.)

The default stack size on kernels with mmu is generally 8 megabytes. Common stack sizes on nommu range from 4k to 128k. Once upon a time toybox could MOSTLY be made to work with 16k, although it's been a while since I've regression tested it and the shell ain't gonna be happy. I'd do more like 64k.

NOMMU also uses its own executable formats, so you have to compile and link your binaries differently.

You can't run most normal ELF binaries on NOMMU, because those are linked to use absolute addresses, and if those specific memory addresses aren't available (because the kernel's there, for example) it won't load and can't run. You MIGHT be able to run ONE ELF binary if you prepared it specially, but can't run even a second copy of the same one (because it would want to re-use the same addresses).

You can run PIE binaries on NOMMU, but it's not ideal. Those glue all the ELF segments together into a contiguous chunk of memory, but have a register pointing to the starting address. So you can run the binary, assuming you can find a big enough contiguous chunk of memory to stick it in. (The heap and stack can fit anywhere, those always had a register pointing to the start of each, even back on x86.) But you need a BIG contiguous chunk, which is hard to find once memory gets fragmented. And it's wasteful because the read-only sections (text and rodata) can't be shared, since they're contiguous with the writeable sections. (There's only one "start of PIE" pointer, and all 4 ELF segments are glued together after it because the offsets are calculated the way they are in conventional ELF, just with the addition of a base pointer instead of always starting at 0.)

FDPIC is an ELF variant that separates the segments, which means it uses 4 registers, one for each segment. (In theory you could use one register pointing to an array of 4 pointers and have an extra dereference on every memory access, but nobody wants to take the speed hit. That's why "fdpic for 32 bit x86" didn't happen.) This means your read only sections CAN be shared, and that the other writeable segments are independently moveable and can fit into smaller scraps of fragmented memory.

SO: cycling back to the original question:

The GLOBALS() block is the majority of the data segment. That and the stack are the two big contiguous allocations for each new process. (The heap is also there but it's happens later and there's stuff libc can do to make it suck less. In theory EACH malloc() could be a separate mmap() which avoids fragmentation and would also round up each allocation to 4k so is actually a huge net loss, but the point is a nommu-aware allocator can break the heap up into multiple mmap()'s to mitigate fragmentation, and that can happen transparently within libc and not be the userspace programmer's immediate problem.)

(Yes I said the kernel gives you a starting heap when I listed the 6 segments above. It's a legacy thing from the PDP-11 days in the 1970s. "man 2 brk" is related. I can explain, but... ask Rich Felker. I he has a rant. I believe this is another thing the libc author can shoot in the face and make it stop; for our purposes you can probably ignore it. Modern allocation is more or less built on mmap(), I think.)

> *As long as "this" is as big as the largest GLOBAL struct, then what is the
> point of working to reduce the global space of any command, when the space
> for "ip" is in there whether "ip" is compiled into toybox or not?*

The space for "ip" shouldn't be there when ip isn't compiled in. Looks like I should add USE() macros around each struct in union global_data in generated/globals.h.

> *What am
> I missing? Why are global structs included in globals.h for commands not
> included in a build? Or are they somehow suppressed in the build?*

On a system with mmu the pages are demand faulted so the main issue there is memory usage being rounded up to 4k.

> *The globals do not seem to affect the size of the executable file, at
> least using the default build. Is the issue with "this" taking up space at
> runtime? Many commands must surely allocate much more space dynamically
> anyway.*
>

> I ask because I have been spending effort to reduce global usage in a toy
> I'm working on, and did some rather large changes of static structs to
> pointer-to-struct to reduce global from 960 to 336, saving 624 global
> bytes, but the code size increased by 285 just due to accessing via
> pointers in many places.

Probably not a net win.

> I don't yet know if that has impacted performance
> noticeably. I am trying to understand if I should back out these changes
> before doing more work.

I err on the side of simple, and clean it up after the fact when I notice a problem. Send the simple thing first.

Part of the reason it's good to have context is if diff is using 456 bytes, anything else trying to get its usage down UNDER that without first addressing diff isn't really helping.)

Every segment actually gets rounded up to multiples of 4k on all the linux systems I'm aware of. (In theory there are 1k allocators, in practice most ext2 filesystems can't be mounted on them.) But "this" isn't the ONLY thing in there, hence scripts/probes/findglobals:

```
$ scripts/probes/findglobals
D 8      toybox_version
B 80     toys
B 4096   libbuf
B 4096   toybuf
D 7648   toy_list
B 8232   this
```

Which is a filtered view of:

```
$ nm --size-sort generated/unstripped/toybox | grep '[0-9A-Fa-f]* [BCDGRS]'
```

0000000000000004	B	__daylight@@GLIBC_2.2.5
0000000000000008	B	__environ@@GLIBC_2.2.5
0000000000000008	B	stderr@@GLIBC_2.2.5
0000000000000008	B	stdin@@GLIBC_2.2.5
0000000000000008	B	stdout@@GLIBC_2.2.5
0000000000000008	B	__timezone@@GLIBC_2.2.5
0000000000000008	D	toybox_version
0000000000000050	B	toys
0000000000000100	B	libbuf
0000000000000100	B	toybuf
00000000000001de0	D	toy_list
00000000000002028	B	this

(But you'll notice all the symbols that got filtered out by the `grep -v GLIBC` are 8k BSS entries. So far, anyway, I dread each upgrade...)

So we have 7648 bytes `toy_list` (just under 2 pages), plus 8 bytes `toy_version` (I'm working to make this a normal string and thus rodata, Elliott has a patch that moves it to rodata but it's still a separate named symbol and thus still CLUTTER that I have to EXPLAIN.)

Anyway, 2 pages of data segment per process.

Meanwhile BSS is also a per-process contiguous memory allocation, so adding up:

4096+4096+8232 is just over 4 pages so need 5 contiguous pages (20480 bytes), and the 80 bytes of "toys" + the glibc debris doesn't change that (goes a trivial amount into that extra page).

So to launch a new toybox instance on a nommu system, I think I need:

1) 20480 bytes

- 2) 8192 bytes data
- 3) 32768 bytes stack.
- 4) unknown heap.

Each of which is a separate allocation, so the high water mark of contiguous memory usage is still the stack, and everything else together roughly doubles the stack. (Modulo what the code then DOES...)

Rob

-
- Next message (by thread): [\[Toybox\] Impact of global struct size](#)
 - **Messages sorted by:** [\[date\]](#) [\[thread\]](#) [\[subject\]](#) [\[author\]](#)
-

[More information about the Toybox mailing list](#)